

-我的方案

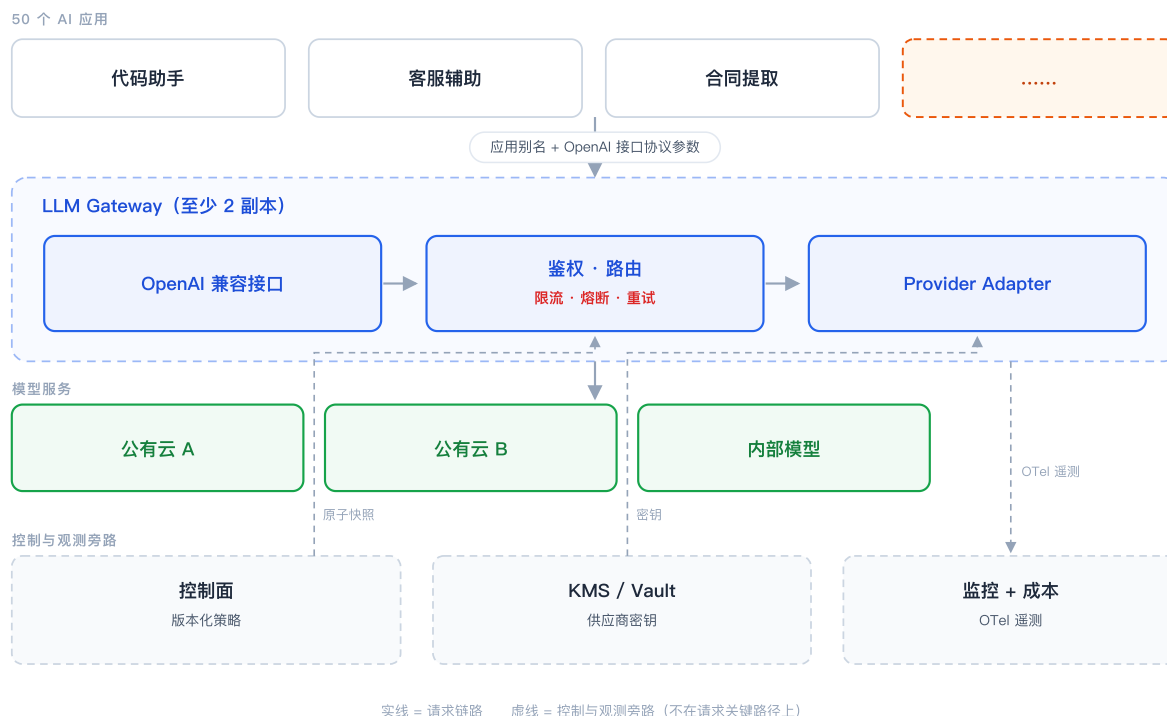
在 50 个应用和多个模型供应商之间，放一个独立无状态的 LLM Gateway。应用只加传"应用名称"这样的业务别名，不关心背后是哪个供应商、密钥在哪。路由、鉴权、限流、故障切换、用量统计全在网关搞定，3 人团队用 LiteLLM Proxy 验证交付，不重复造轮子。

背景：增长带来的管理债

一家企业过去一年陆续上线了 20 多个 AI 应用：内部知识助手、客服辅助、合同提取、代码助手、数据分析。这些应用由不同团队各自开发，分别调着两家公有云模型和一套内部部署的开源模型。日均请求约 30 万次，峰值 100 QPS，既有同步调用也有 SSE 流式。

规模不大，但已经能闻出问题的味道：不同团队各自写 Adapter、各自管密钥、各自搞重试和流式处理。没有一个地方能看清全公司的模型用量和费用。某个供应商限流了，十几支团队各修各的。公司打算一年内把应用扩到 50 个，再不收口，管理成本迟早高过调用成本。

整体架构



所有应用只需一套 OpenAI 兼容接口，传应用身份这样的业务别名。供应商型号、地址、密钥全在网关背后——应用不知道也不该知道。Prompt 编排、会话管理和结果投递不进网关，网关只管"怎么调"，不管"拿模型做什么"，应用可以在控制面选择模型的优先级。

为什么是独立网关

先说走过的一条近路。

之前我们用过一个更轻的方案：内嵌 Provider。每个应用里注册一个 Provider 实现，通过枚举映射选模型，Reactive Stream 直连模型 SSE，再用双通道（SSE 推送 + 缓存轮询）把结果送给前端。十几个应用、百 QPS 以内，这套跑得很顺——代码干净、链路短、没有额外组件要运维。

但应用往 50 个去的时候，几个问题同时冒头：

- 每上线一个新应用，就得把 Adapter 代码再拷一遍，Provider 实现越积越多；
- 密钥散在十几个项目的配置文件里，安全团队想做一次全量审计，光找密钥在哪就花了一天；
- 供应商要下线一个旧模型、换一个新模型，十几个团队的改造排期横跨三周；
- 用量和费用靠各应用自己上报，有的报了有的没报，月底财务拉账单跟供应商对，永远有缺口。

这些不是技术 bug，是规模到了、架构没跟上的管理债。应用越多，内嵌模式的维护成本涨得比调用量还快。

所以这次的方案很明确：加一层薄网关。

应用侧该怎么写业务还怎么写，把“调哪个模型、怎么鉴权、限多少流、出错怎么办”这些事从每个应用里抽出来，在网关统一解决。

网关只管四件事

统一入口。 所有应用面对同一套接口。调用方只知道业务别名——chat-balanced 日常对话、extract-lowcost 批量提取。具体映射到哪个供应商的哪个型号，在网关策略里配置，应用不关心。新增供应商或切换模型时，只改网关策略，不改应用代码。

集中治理。 路由、鉴权、配额检查、限流、重试逻辑都在网关数据面完成。每个请求进来就绑定一个策略快照——从鉴权到返回结果，同一次调用用同一套规则，不会出现流式推到一半策略被人改了的情况。

故障隔离。 这是最核心的一条：首字节发给用户之前，超时了、限流了、5xx 了——可以重试，也可以切同级别的备选模型。但首字节一旦推送到用户屏幕上，再出问题只能发“出错了”然后结束。不能偷换一个模型接着输出——两个模型的碎片拼在客户端，没人说得清是谁说的、哪个是对的。

透明计量。 一个业务请求，网关背后可能转了多次上游调用（重试、切换各算一次）。业务层记一次，上游层记每次，两本账对得上。成本按调用时的单价快照算，每天跟供应商账单比对，差异超过 3% 当天排查原因。

三个选择

选 LiteLLM，不自研。 业界目前三条路：LiteLLM（内置虚拟 Key、模型别名、预算限流、路由回退，五分钟部署）；Envoy AI Gateway 和 Higress（基于 Envoy 扩展，适合有 K8s 运维底盘的团队）；完全自研。3 人团队走自研，短期看上去可控，长期维护的坑比开发还深——适配新模型、修 bug、跟进安全补丁，每一项都是持续成本。以 LiteLLM 做验证基线，但北向接口和策略模型保持产品中立——将来觉得 LiteLLM 不够用了，换 Envoy 或 Higress，应用侧不用动。

控制面不进入热路径。 策略变更在控制面做，校验后打成原子快照推到数据面。控制面挂不影响在线调用——数据面用缓存的有效策略继续跑。快照带有效期，过期了就拒绝，不留旧密钥和旧路由挂在线上。

首包是重试的硬边界。 这是实打实踩过的坑。数据没到用户手里，怎么重试怎么换都可以。首包一旦推送出去，再换模型意味着用户看到的是多个模型的混合输出——既不可审计，也无法保证语义连贯。LiteLLM 和 Envoy AI Gateway 在路由回退设计上也是同样的原则。

代价

多一跳延迟。同步请求 P95 多 50ms 以内，SSE 首字 P95 多 100ms 以内——这要靠同地域部署、连接池复用和就近解析才能兑现，不是白送的。

网关自己是新的故障点。所以至少双副本部署，做好舱壁隔离和滚动排空。每个应用还得保留直连的后路，网关全挂时能一键切回。

什么情况不适用。

应用不到 10 个，一天几万次调用，就一两个供应商——内嵌 Provider 够用，少一跳、少一个组件，更省心。20 个应用、3 类供应商的时候，网关开始赚回票价。50 个应用基本是必需品。如果未来模型间协议差异大到收不拢（gRPC 流、二进制帧），统一接口的抽象会被逐步撑破，那时网关模式也得重新评估。

本方案不覆盖训练、微调、RAG 和 Agent 编排。也不承诺不同模型输出等价——回退只在同能力、同数据等级的候选模型间发生，对话质量验收仍是各应用自己的责任。